

ONERA 468 CRM Challenge: Wall Distribution Prediction

Oualid Chabane

LISN, Université Paris-Saclay, Master 1 AI & Magistère 2
Internship in collaboration with ONERA, supervised by L. Sun and S. Chevallier

Abstract

We present the design of a second version of a machine learning challenge built on 468 RANS CFD simulations of the NASA/Boeing Common Research Model (CRM) [1], [2] aircraft, as a continuity upon the first version of the challenge previously released. The challenge targets the prediction of the surface volumetric density field ρ from flow conditions and aircraft geometry, with the goal of evaluating a model's capacity to understand the underlying physics rather than merely interpolating. We describe a Mach-stratified train/test split constructed to separately evaluate interpolation and extrapolation of the participants' models, and we propose a component-weighted Kullback-Leibler divergence as a primary metric, and Pearson's correlation and the worst absolute error as secondary metrics. Two main baselines that represent interesting directions to build powerful models are provided, which are a POD+Gaussian-Process model and a full-field MLP trained directly on a differentiable relaxation of the KL metric.

1. Introduction

The Reynolds-Averaged Navier-Stokes (RANS) Computational Fluid Dynamics (CFD) simulations central to aircraft design remain computationally expensive; machine-learning surrogates are an attractive way to make this analysis fast enough for iterative design, provided the model can be trusted to generalize rather than merely interpolate. This report describes a machine learning challenge, designed during an internship at LISN in collaboration with ONERA and hosted on Codabench [3] (platform details in Section A.6), that targets prediction of the non-dimensional wall density field ρ/ρ_∞ over the NASA/Boeing Common Research Model (CRM) [1], [2] from 468 RANS simulations (260,774 surface points each), given the aircraft geometry and three flow conditions: freestream Mach number M_∞ , angle of attack α , and stagnation pressure Π (full dataset and physics background in Section A.1).

The central design question is whether a model has learned the underlying physics rather than the training distribution. Two choices address this directly: a train/test split that separately evaluates interpolation and extrapolation (Section 2.1), and a primary metric that targets the **shape** of the error distribution rather than a single aggregate number (Section 2.2). Two main baselines illustrate promising directions for participants: POD+Gaussian-Process (POD+GP) and a full-field MLP trained on a differentiable relaxation of the metric (MLP_{KLW}); these and four other baselines are compared in Section 4 on a rebalanced Mach-stratified split.

1.1. Related Work

Existing benchmarks for aerodynamic ML surrogates typically evaluate models purely in-distribution. AirfRANS [4], the closest precedent, does separate in-distribution and out-of-distribution test conditions (Reynolds-number and angle-of-attack extrapolation regimes), which motivated the two-phase structure adopted here, and evaluates with Spearman correlation and root mean squared error. This challenge differs by targeting a 3-dimensional full-surface scalar field rather than a 2D airfoil, and by scoring the **shape** of the error distribution via a Kullback-Leibler divergence to a reference distribution rather than a first- or second-moment aggregate; the general framing of intrinsic versus learnable difficulty follows [5].

2. Methods

2.1. Train/Test Split Design

A challenge’s difficulty decomposes into an intrinsic part (data noise and flaws) and a learnable part (recoverable by any model) [5]; the goal is a split that maximizes the ratio of the latter to the former. We test both interpolation and extrapolation, since a model that generalizes to unseen flow conditions must have captured genuine structure rather than exploited local smoothness, particularly given that the flow-condition distribution clusters densely in several regions (e.g. $M_\infty \in [0.8, 0.9]$), where locality-based methods such as KNN can otherwise score deceptively well on interpolation splits (see Section A.5). Analysis of ρ/ρ_∞ against each flow-condition axis showed the distribution’s parameters are clearly correlated with M_∞ but not with α or Π , motivating a Mach-stratified split (Table 1): the excluded interpolation band (Phase 1) sits inside the dense training cluster, while the extrapolation band (Phase 2) sits at the distribution’s extremes, entirely outside the training range.

Split	Mach numbers	# simulations	% of 468
Train	0.50, 0.70, 0.75, 0.80, 0.82, 0.85, 0.88, 0.90, 0.93	324	69.2%
Phase 1 (interp.)	0.84, 0.86	72	15.4%
Phase 2 (extrap.)	0.30, 0.96	72	15.4%

Table 1: Train/test split employed in this challenge.

2.2. Evaluation Metrics

Pearson’s R^2 and a worst-case relative MAE (wrMAE) are used as secondary metrics, but neither is sensitive to **localized, structured** errors: a model can score near-perfectly overall while systematically mispredicting a small, physically important region (e.g. the pylon), since both metrics average over all points (full formulas in Section A.2). Figure 1 illustrates the failure mode this misses: a dominant peak centered at 0 with a secondary peak offset from zero, revealing a systematic, localized bias that averaging-based metrics do not detect.

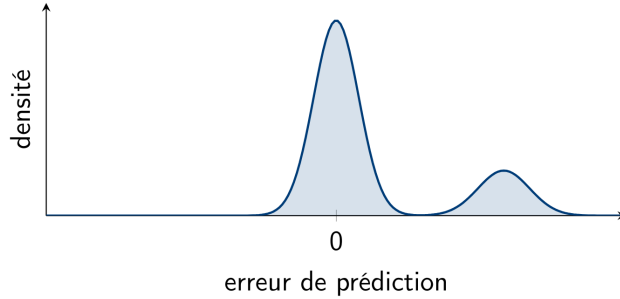


Figure 1: An error distribution that has to be penalized severely: a dominant peak at 0 plus a secondary, offset peak revealing a localized bias.

The primary metric instead targets the **shape** of the per-simulation residual distribution via a Kullback-Leibler divergence to a centered reference Gaussian $\mathcal{N}(0, 0.01\bar{\rho})$, computed separately per aircraft component (wing, pylon, fuselage, nacelle) and combined with weights $w_{\text{wing}} = w_{\text{pylon}} = 0.3$, $w_{\text{fuselage}} = w_{\text{nacelle}} = 0.2$ that emphasize the two hardest-to-predict components:

$$\text{KL}_i^w = D_{\text{KL}}(P_i^w \parallel \mathcal{N}(0, 0.01\bar{\rho}_i)), \quad \overline{\text{KL}}^w = \frac{1}{N} \sum_{i=1}^N \text{KL}_i^w$$

$\overline{\text{KL}}^w$ (denoted **mean_KL**) is the leaderboard’s primary, rank-determining metric (lower is better). Because the histogram underlying it is not differentiable, MLP_{KLW} (Section 3) is instead trained on a soft-binned relaxation of the same objective, replacing the hard bin assignment with a softmax over bin centers; the full derivation, including the exact PyTorch tensor shapes of the resulting rank-3 backpropagation, is given in Section A.2.1.

3. Baselines

POD+GP. Training density fields are reduced via Proper Orthogonal Decomposition (POD, equivalently PCA) to $k = 55$ retained modes; one independent Gaussian Process (Matérn, $\nu = 1.5$) is fit per mode, mapping (M_∞, α, Π) to that mode’s coefficient, and the field is reconstructed via the **exact linear** POD decoder $\hat{\rho} = \bar{\rho} + \sum_{m=1}^k z_m \varphi_m$. A Gaussian Process is chosen over a fixed interpolator precisely because the kernel it is built on decides which functions are plausible a priori: an RBF/Matérn kernel forces nearby flow conditions to covary and yields smooth, physically plausible mode coefficients, whereas an unconstrained interpolator has no such prior (Figure 2).

Hyperparameters were tuned via a 181-trial Optuna search (Section A.4); a full primer on Gaussian Processes and their link to this decoder is given in Section A.3.

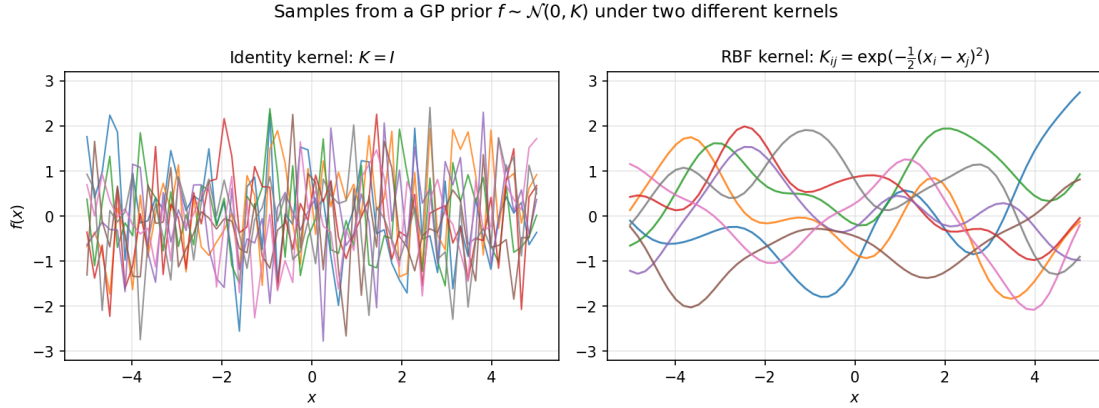


Figure 2: Samples from the same prior $f \sim \mathcal{N}(0, K)$ under two kernels: identity (left, uncorrelated noise) versus RBF (right, smooth, plausible functions). POD+GP’s per-mode GPs use a Matérn kernel, closely related to the RBF case.

MLP_{KLW}. A full-field MLP (hidden layers 128, 256, 512) trained directly on the differentiable KL relaxation of Section A.2.1, serving as a direction for participants to explore deep architectures rather than as a tuned, final model.

Other baselines (full descriptions and tuning in Section A.4):

- A **global KNN regressor** ($k = 5$ nearest neighbors on the scaled flow conditions alone, geometry not used at all) predicts the entire field as the average of the 5 nearest training simulations. It scores deceptively well on interpolation by proximity ($R^2 = 0.9824$ on Phase 1) but collapses on extrapolation, since it never learns the actual dependence on (M_∞, α, Π) , only how to look up the closest match.
- A **mean predictor** simply predicts the global training mean, as a sanity floor every other baseline should beat.
- **IsoMap+RBF**, the best-performing model from [2], embeds fields into a 3D latent space via IsoMap and maps flow conditions to that space with an RBF interpolator; reconstruction backmaps via an inverse-distance-weighted average of the k nearest training fields in latent space, a convex combination that therefore cannot exceed the training range.
- A **literature global MLP** [2] (hidden layers 75, 120, 1226, 16490), trained with plain MSE rather than a KL-aware loss; the top-ranked architecture in the predecessor challenge, evaluated here only on the Margaret cluster due to Codabench resource limits.

4. Results and Discussion

Table 2 reports every baseline on both phases of the rebalanced split; **mean_KL** is the rank-determining metric (lower is better). All values are live Codabench scores except Global MLP, evaluated only on the Margaret Inria supercomputer.

Model	Phase	R^2	wrMAE	mean_KL	Score
Mean	1	-0.0039	0.1946	13.817	4.007
KNN	1	0.9824	0.0235	1.314	9.794
KNN	2	0.8480	0.0982	7.410	8.749
IsoMap+RBF (submitted)	1	0.9864	0.0334	0.849	9.765
IsoMap+RBF (submitted)	2	0.8431	0.0708	6.576	8.862
POD+GP (submitted)	1	0.9869	0.0287	0.881	9.791
POD+GP (submitted)	2	0.9233	0.0650	4.551	9.292
MLP _{KLW} (submitted)	1	0.927	0.0459	1.92	9.405
MLP _{KLW} (submitted)	2	0.850	0.0872	4.93	8.812
Global MLP (local only)	1	0.9881	0.0304	0.754	9.789
Global MLP (local only)	2	0.8892	0.0729	5.170	9.081

Table 2: Baseline comparison on Phase 1 and Phase 2. **mean_KL** is the primary, rank-determining metric (lower is better); **Score** = $5R^2 + 5(1 - \text{wrMAE})$ is secondary. Bold marks the best mean_KL per phase, including the locally-evaluated Global MLP result.

On Phase 1 (interpolation), Global MLP, IsoMap+RBF, and the global KNN regressor all edge out POD+GP, KNN in particular scoring $R^2 = 0.9824$, mean_KL= 1.314, despite never seeing the aircraft’s geometry, an artifact of the Phase 1 Mach values (0.84, 0.86) sitting tightly bracketed by dense training values (0.82, 0.85, 0.88). Phase 2 exposes this: at $M_\infty \in \{0.30, 0.96\}$, far outside the training range, KNN’s mean_KL collapses to 7.410 (full analysis in Section A.5). POD+GP has the best Phase 2 mean_KL of every baseline (4.551, submitted or not), consistent with its exact linear decoder versus IsoMap+RBF’s backmapping, which cannot, by construction, extrapolate past the training range; swapping POD+GP’s Gaussian Process for a plain RBF interpolator over the same POD basis (the configuration used in [2]’s own baseline) likewise underperforms, echoing the winning MMGP architecture of the NeurIPS 2024 ML4CFD competition [6], which similarly pairs a reduced representation with a Gaussian Process rather than a fixed interpolator. Since Phase 2 determines the final rank, POD+GP is the strongest baseline overall.

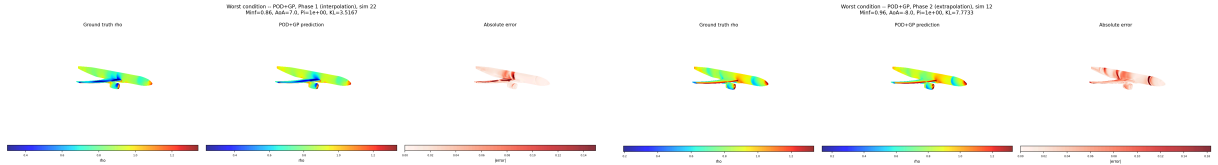


Figure 3: POD+GP: worst-KL condition, ground truth / prediction / absolute error. Left: Phase 1. Right: Phase 2.

Figure 3 shows the worst-KL condition on each phase; the error stays small and diffuse in Phase 1 but grows in a structured, localized band in Phase 2, consistent with extrapolation error concentrating where the model has the least support from training data rather than degrading uniformly. Full per-model error diagnostics (additional heatmaps and error-weighted PCA slices, for both POD+GP and MLP_{KLW}) are given in Section A.5.

5. Conclusion

This internship produced a machine learning challenge, hosted on Codabench, for predicting the surface density field of the CRM aircraft from RANS simulations. Two main baselines are proposed to the participants, which are the Gaussian Process preceded by Proper Orthogonal Decomposition,

and a second baseline which is a Multi-Layer Perceptron with the weighted Kullback-Leibler metric as an objective. Other interesting models are proposed.

The train/test split that was proposed is organized in two phases: the first phase tests interpolation whereas the second phase tests extrapolation. This split helps significantly to push models towards learning the underlying physics. The proportions of the training data to the testing data are respected in order to make the challenge comparable to its prior version.

Another metric was proposed to assess models' sensitivity to localized errors, which is a KL-divergence-based metric. The other metrics are retained. The primary metric is to be decided later, but for the moment it is the KL-divergence-based metric. On the Codabench split, POD+Gaussian-Process achieves the best extrapolation-phase performance on the primary metric among all baselines. Future work includes an extension toward force-based physical validation (drag, lift), which is a more physically grounded evaluation axis.

A Appendix

A.1 Dataset and Physics Background

In the challenge the participants are given 468 RANS simulations of the NASA/Boeing Common Research Model (CRM), each simulation described by 260,774 surface points, and the corresponding labels which represent the wall volumetric density field ρ . The goal is to build a model that beats the other participants’ models on the primary metric. The input features of the datasets are given (in order) by:

- 3 dimensions for the Cartesian coordinates (x, y, z) of each surface point,
- 3 dimensions for the unit normal vector at each surface point,
- 3 dimensions for the flow conditions:
 - the freestream Mach number M_∞ : the ratio of the aircraft speed to the speed of sound.
 - the stagnation pressure Π : the pressure of the flow brought to rest isentropically, setting the overall energy level of the simulation; in simpler words,

it represents the pressure of the chamber right before the air starts flowing.

- the angle of attack α : the angle between the freestream velocity and the aircraft chord line (see Figure 5).

Notes:

- The aircraft geometry is fixed (ex. we don’t have wing movements caused by air flow). Furthermore, all simulations are captured in one state, meaning, there is no time evolution in the dataset.
- Stagnation pressure: imagine a small parcel of moving fluid. It has some pressure (static pressure) and some kinetic energy from

its velocity. Now imagine you slow that parcel down to zero velocity without any losses (no friction, no heat transfer, no shock waves), entropy remains the same (that’s what “isentropically” means). As the fluid decelerates, its kinetic energy has to go somewhere; by energy conservation principle, it converts into pressure (and temperature) rise. The pressure you’d measure once the fluid is fully at rest, having lost only its motion and nothing else, is the stagnation pressure P_i .

Table 3 and Figure 4 show, respectively, how many simulations we have for each flow condition value, and what the aircraft geometry looks like (the same for all simulations).

Flow condition	Distinct values	Range	Simulations per value	Total
Mach number M_∞	13	0.30 to 0.96	36 (uniform)	468
Stagnation pressure Π	3	1, 2, 4 Pa	156 (uniform)	468
Angle of attack α	34	-15° to 15°	6, 6, 6, 12, 15, 12, 12, 9, 18, 21, 12, 15, 12, 12, 9, 39, 12, 15, 18, 6, 27, 27, 6, 27, 24, 9, 12, 18, 15, 12, 6, 6, 6, 6	468

Table 3: Number of simulations per discrete value of each flow condition axis. Mach number and stagnation pressure are sampled on a uniform full-factorial grid (36 and 156 simulations per value respectively); angle of attack is sampled more densely near 0° , with per-value counts listed in ascending order of α (peak of 39 at 0°).

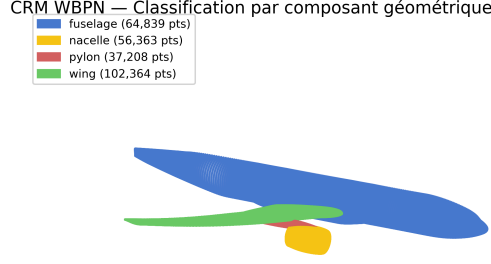


Figure 4: Geometry visualization with to-components segmentation.

From a physical standpoint, four main forces act on an aircraft in flight: **lift**, the force perpendicular to the freestream velocity that supports the aircraft against gravity; **drag**, the force parallel to it that opposes the motion; **thrust**, the force opposite to the drag, generated by the engines; and **weight**. See Figure 5 for more details.

During takeoff, the aircraft accelerates along the runway, increasing its speed and therefore increasing the airflow over the wings. The form of the wings makes the air particles travel faster over the extrados than over the intrados; this difference induces a difference in pressure between these two regions, which makes the lift force increase. Once lift exceeds the aircraft's weight, it leaves the ground. During landing, the process is reversed: the pilot reduces engine thrust and increases the angle of attack α , which raises drag and reduces speed, causing lift to drop below the aircraft's weight until it touches down.

ρ is a very important quantity to predict, because it figures directly in the Navier-Stokes equations:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0$$

$$\frac{\partial(\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{u} \otimes \mathbf{u}) = -\nabla p + \nabla \cdot \boldsymbol{\tau} + \rho \mathbf{f}$$

Rather than predicting the raw density ρ , the challenge targets the non-dimensional ratio ρ/ρ_∞ where ρ_∞ is the stream density at rest state. A value of $\rho/\rho_\infty = 1$ indicates undisturbed flow, values above 1 indicate local compression, and values below 1 indicate local expansion. Dividing by ρ_∞ removes the dependence on the absolute flow conditions, making the field comparable across all simulations regardless of their operating point (see Figure 8, Figure 7, Figure 6).

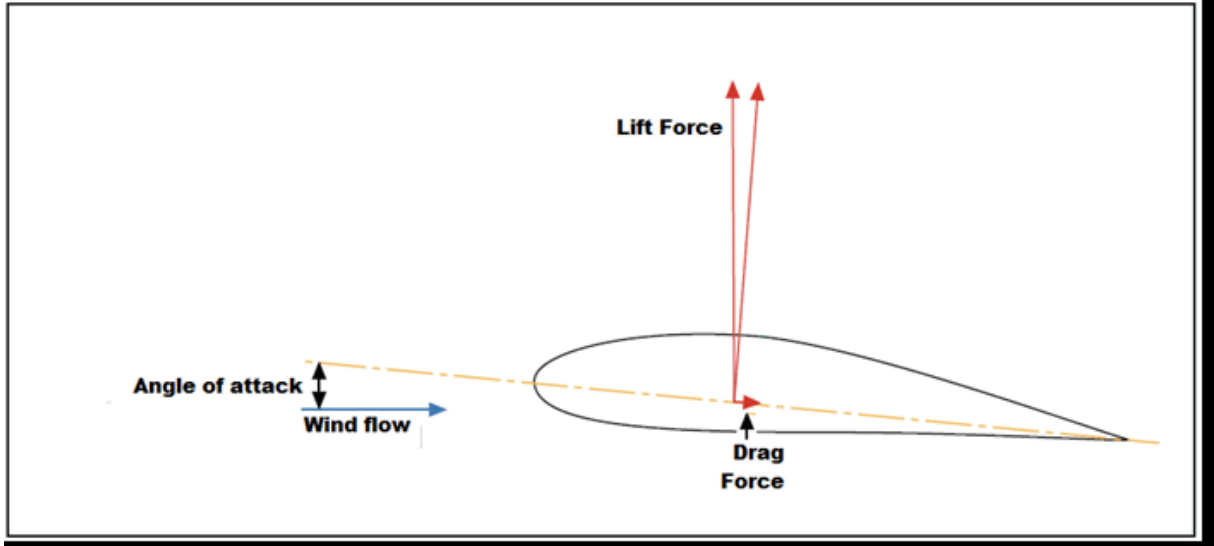


Figure 5: Schematic of the aerodynamic forces acting on the aircraft. Lift acts perpendicular to the freestream velocity, drag acts parallel to it, and the angle of attack α controls the orientation of the aircraft relative to the incoming flow.

The dataset contains two types of simulations depending on the convergence behavior of the Spalart-Allmaras turbulence model: **well-converged simulations**, where the standard deviation of the measured lift and drag forces across solver iterations remains small, assigned a confidence weight $w_f = 1.0$; and **low-confidence simulations**, where this standard deviation is large, indicating the solver did not fully stabilize, assigned $w_f = 0.5$. Practically, when the absolute value of the angle of attack exceeds 10, we consider the simulation ill-converged.

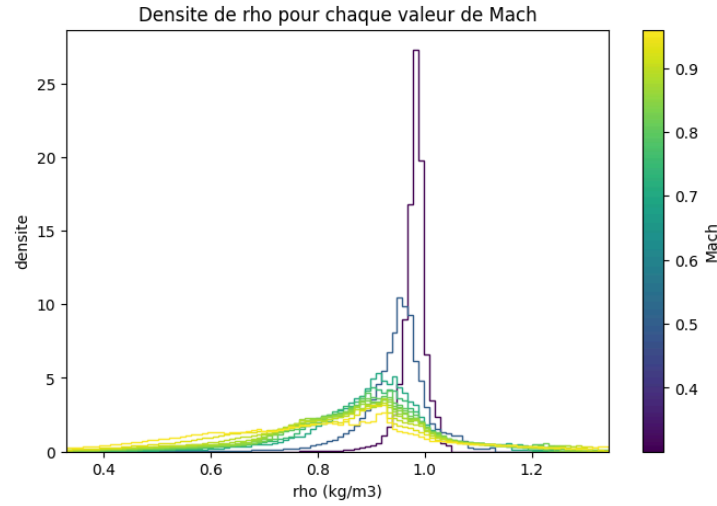


Figure 6: Distribution of ρ/ρ_∞ as a function of M_∞ .

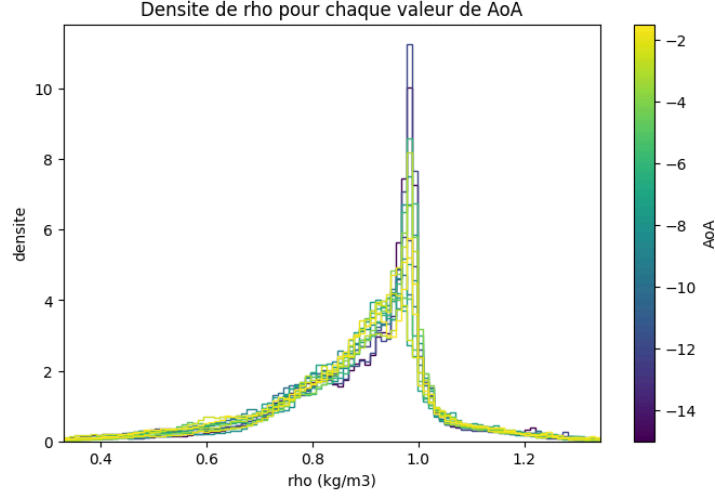


Figure 7: Distribution of ρ/ρ_∞ as a function of angle of attack α .

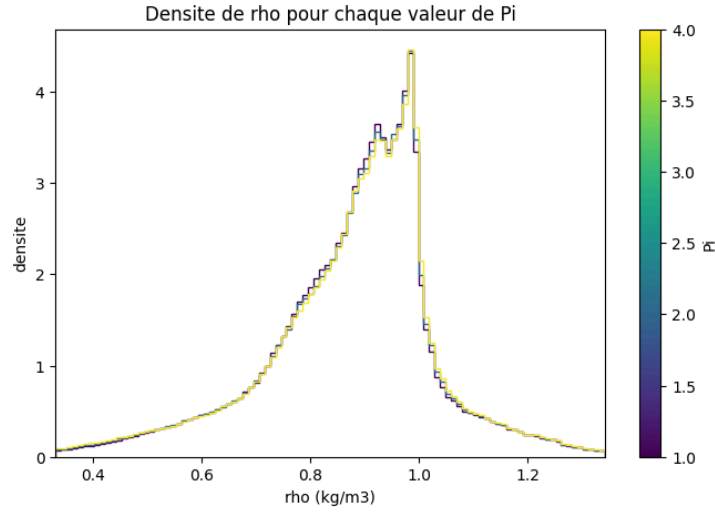


Figure 8: Distribution of ρ/ρ_∞ as a function of stagnation pressure Π .

The work currently completed contains five main milestones:

- **Milestone 1:** Creation of the first draft of the challenge on Codabench, covering the ingestion and scoring pipeline, support for external packages beyond the base Docker image, and memory handling since we have a large dataset. This milestone was not concerned with model selection, metrics, or dataset analysis. The train/test split was defined along the Mach number axis without rigorous justification.
- **Milestone 2:** A more rigorous study of the train/test split, revisiting all possible configurations and evaluating whether machine learning models can generalize from the training distribution to the test set.
- **Milestone 3:** Reviewing the metrics initially proposed by colleagues at ONERA and investigating other metrics to steer models toward physically meaningful predictions.
- **Milestone 4:** Mid-internship meeting, updating the challenge with the resulting feedback and starting to explore the models.
- **Milestone 5:** Design, tuning (via Optuna Bayesian search where applicable) the best models, updating the Codabench platform to fit the final changes and upgrades.

A.2 Evaluation Metrics: Full Derivation

Two standard metrics initially used by the colleagues of ONERA are the Pearson’s correlation coefficient R^2 :

$$R^2 = 1 - \frac{\sum_f w_f \sum_{i=1}^{n_p} (\rho_i^f - \hat{\rho}_i^f)^2}{\sum_f w_f \sum_{i=1}^{n_p} (\rho_i^f - \bar{\rho})^2}$$

where f indexes the test simulations, i indexes the surface points ($n_p = 260,774$), $w_f \in \{0.5, 1.0\}$ is the confidence weight of simulation f (1.0 for well-converged, 0.5 for low-confidence), and $\bar{\rho} = \frac{\sum_f w_f \sum_i \rho_i^f}{\sum_f w_f \cdot n_p}$ is the weighted mean density across all test points and simulations; and the weighted worst-case Mean Absolute Error (wrMAE), computed in two steps. First, for each simulation f , the relative MAE is:

$$\text{rMAE}^f = \frac{\sum_{i=1}^{n_p} |\rho_i^f - \hat{\rho}_i^f|}{\sum_{i=1}^{n_p} |\rho_i^f|}$$

Then the wrMAE retains only the worst-performing simulation among the well-converged ones (those with $w_f = 1.0$, i.e. $|\alpha| < 10^\circ$):

$$\text{wrMAE} = \max_{f \in \mathcal{W}} \text{rMAE}^f$$

where $\mathcal{W}$ is the set of well-converged test simulations.

Limitations: These metrics are not sensitive to **localized, structured** errors, which are very frequent in our case. A model can achieve a near-perfect R^2 or a near-zero wrMAE while still systematically mispredicting ρ on a small but physically important region (e.g., the pylon or along the wing). This is the case for the global KNN regressor, which scores near-perfect values, but after plotting the error curve, we find high peaks in the previously mentioned regions. This gap between the results and the real effect is due to the averaging present in both previously stated metrics; this averaging kills the fluctuations. The type of error distribution we want to penalize is illustrated in Figure 9: a dominant peak centered at 0 and a secondary peak offset from zero, revealing a systematic bias in a localized region, something that R^2 and wrMAE both fail to detect due to averaging.

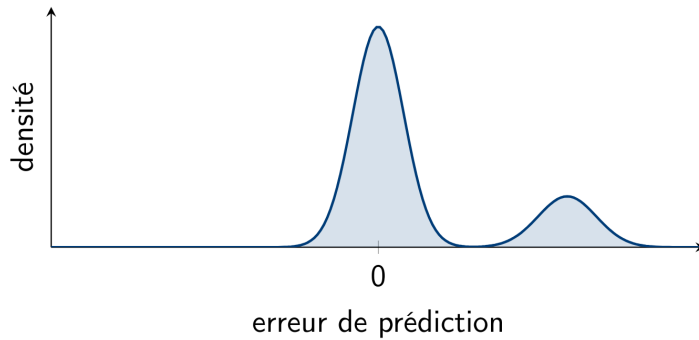


Figure 9: Illustration of an error distribution that has to be penalized severely.

To address this, we designed a metric based on the Kullback-Leibler (KL) divergence between the empirical distribution of prediction errors P and a reference which is in our case a centered Gaussian distribution $Q = \mathcal{N}(0, 0.01\bar{\rho})$, i.e. a standard deviation set to 1% of the mean ρ for the simulation in question (in other words, the closer the deviation of the error is to 1% of the average value of what we are predicting, the better our model is):

$$D_{\text{KL}}(P \parallel Q) = \int_{-\infty}^{\infty} p(x) \log\left(\frac{p(x)}{q(x)}\right) dx$$

The closer to 0 the better it is. For each test simulation i , the KL divergence KL_i between its error distribution and the reference Gaussian is computed, and the final score (unweighted version of the primary metric) is given by the average of KL_i over all N test simulations:

$$\overline{\text{KL}} = \frac{1}{N} \sum_{i=1}^N \text{KL}_i$$

Note: Computing the KL divergence on the total error of all simulations to a centered Gaussian has no sense physically speaking since it merges all the simulations together, furthermore, it makes it harder to detect these hard regions.

We noticed that in most cases, the models fail to predict well on the wing and on the pylon more than the other components of the aircraft; these regions are the most interesting ones, therefore, we considered a weighted version of the Kullback-Leibler divergence as the following, the aircraft surface is decomposed into four components, wing, pylon, fuselage, and nacelle and the weighted metric is given by:

$$\text{KL}_i^w = D_{\text{KL}}(P_i^w \parallel \mathcal{N}(0, 0.01\bar{\rho}_i)), \quad \overline{\text{KL}}^w = \frac{1}{N} \sum_{i=1}^N \text{KL}_i^w$$

where $w_{\text{wing}} = w_{\text{pylon}} = 0.3$ and $w_{\text{fuselage}} = w_{\text{nacelle}} = 0.2$.

$\overline{\text{KL}}^w$ (denoted **mean_KL**, or **KLw**, in the results) is the leaderboard’s primary metric, sorted in ascending order (lower is better). R^2 and wrMAE remain displayed as secondary, contextual metrics, alongside the composite secondary score

$$\text{Score} = 5R^2 + 5(1 - \text{wrMAE})$$

(out of 10, higher is better), which does not itself involve the KL divergence and is not what a submission is ranked on.

As a summary:

$$R^2 \in (-\infty, 1] \quad (\text{higher is better})$$

$$\text{wrMAE} \in [0, +\infty] \quad (\text{lower is better})$$

$$\text{Score} \in [-\infty, 10] \quad (\text{higher is better})$$

$$\overline{\text{KL}}^w \in [0, +\infty] \quad (\text{lower is better})$$

A.2.1 A differentiable KL loss for training

The regular KL loss presented earlier cannot be used as a training objective with gradient-based methods, as it is not differentiable. We therefore present the following relaxation of the metric, which is differentiable.

The official **mean_KL** metric relies on `np.histogram`, which performs hard bin assignment. This induces a gradient that is zero almost everywhere. More precisely, we can view the error distribution as a sum of random variables ε_i associated with the error at each skin point, where each ε_i follows a Dirac distribution. Consequently, when we differentiate the histogram, we obtain a zero gradient almost everywhere on the domain, since it is a combination of Dirac distributions, and the histogram is non-differentiable at the bin boundaries.

In order to make it differentiable, we replace the dirac distributions by a softmax distribution, hence it becomes **soft** binning (as the one used in MLP_{KLW} , Section 3). Each residual $\varepsilon_i = (\hat{\rho}_i - \rho_i)$ is given a Gaussian-softmax weight over every one of 200 bin centers c_b spanning $[-5, 5]\sigma_s$ (softmax is applied across the ε axis):

$$w_{i,b} = \text{softmax}_b \left(-\frac{(\varepsilon_i - c_b)^2}{2\tau^2} \right)$$

with $\tau = 0.05\sigma_s$, exactly one bin-width, it's the ideal choice to stay faithful to the true histogram while smoothing to avoid zero-gradients. τ actually is exactly the standard deviation of a distribution of a certain error at a certain bin, the softmax function actually hides a Gaussian distribution behind it (the denominator of the softmax is nothing but the normalizing constant of the Gaussian distribution). Aggregating these soft votes over all points, weighted by the same per-component confidence weight w_i used in the official metric (wing/pylon 0.3, fuselage/nacelle 0.2), gives a smoothed empirical histogram:

$$p_b = \sum_i w_i w_{i,b}$$

Here $p, q \in \mathbb{R}^B$ are plain vectors of length $B = 200$ (the number of bins) where p is the differentiable soft histogram of predicted residuals, q the fixed reference histogram. p is compared to the fixed reference $q_b \propto \mathcal{N}(0, 0.01\sigma_s)$ via ordinary KL divergence:

$$\mathcal{L} = D_{\text{KL}}(p \parallel q) = \sum_b p_b \log \left(\frac{p_b}{q_b} \right)$$

Writing $\theta \in \mathbb{R}^P$ for the network's P trainable weights, S for the batch size (the number of simulations processed per training step, $S = 2$ here) and N for the number of surface points per simulation, the actual PyTorch tensors are not the flat vectors of a single-simulation toy example: ε is rank 2 (one residual per point, per simulation in the batch), w is genuinely rank 3, `torch.Size([S, N, B])` (one softmax weight per point, per bin, per simulation), and p is rank 2 again once the point axis has been summed out. The forward pass has shapes

$$\theta \xrightarrow{\text{net} + \text{residual}} \varepsilon \in \mathbb{R}^{S \times N} \xrightarrow{\text{softmax}} w \in \mathbb{R}^{S \times N \times B} \xrightarrow{\text{aggregate}} p \in \mathbb{R}^{S \times B} \xrightarrow{\text{KL}} \mathcal{L} \in \mathbb{R}$$

The backpropagation chain is given by, writing each tensor's **flattened** size for the Jacobian shapes:

$$\underbrace{\frac{\partial \mathcal{L}}{\partial \theta}}_{1 \times P} = \underbrace{\frac{\partial \mathcal{L}}{\partial p}}_{1 \times (SB)} \underbrace{\frac{\partial p}{\partial w}}_{(SB) \times (SNB)} \underbrace{\frac{\partial w}{\partial \varepsilon}}_{(SNB) \times (SN)} \underbrace{\frac{\partial \varepsilon}{\partial \theta}}_{(SN) \times P}$$

Only $\partial \varepsilon / \partial \theta$, the Jacobian of the network's own output with respect to its weights, requires standard backpropagation through the MLP; the other three factors are cheap, closed-form derivatives of the soft-binning and aggregation steps introduced above, and both middle Jacobians are sparse ($p_{s,b}$ only depends on simulation s 's slice of column b of w , and $w_{s,i,b}$ only depends on $\varepsilon_{s,i}$), so in practice PyTorch's autodiff never materializes these as dense matrices, it walks the same rank-3 tensor operations (`unsqueeze`, `softmax`, `einsum`) backward instead. This is what lets $\mathcal{L}$ be minimized directly by gradient descent, instead of a generic stand-in such as MSE.

A.3 Gaussian Processes

Since POD+GP scores the best results of all baselines on the extrapolation phase, the most critical one (see Section 4), this section gives a short primer on what a Gaussian Process (GP) actually is, following [7].

A GP prior is a multivariate normal distribution over function values, specified entirely by a kernel (covariance) function k :

$$P(f \mid X) = \mathcal{N}(f \mid \mu, K), \quad K_{ij} = k(x_i, x_j)$$

Every entry of the covariance matrix K comes from the same kernel, so the kernel alone decides which functions are plausible under the prior. Figure 10 shows samples drawn from this prior for two very different choices of k : with an **identity kernel** ($K = I$, every point independent of every other), the samples are just uncorrelated noise, jagged and disconnected; with an **RBF kernel**, nearby points are forced to covary, and the samples become smooth, plausible functions of x .

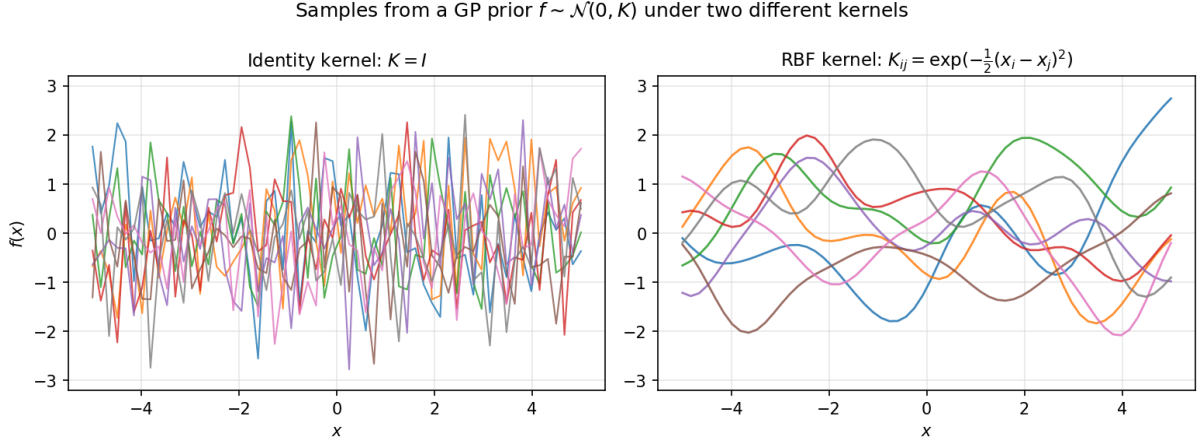


Figure 10: Eight samples from the same 60-variate normal prior $f \sim \mathcal{N}(0, K)$, identity kernel (left) versus RBF kernel (right). Only the RBF prior actually looks like a distribution over functions.

Conditioning on observed data turns this prior into a posterior: the same multivariate normal, rescaled with respect to the training inputs X and labels y . Writing $K = k(X, X)$, $K_* = k(X, X_*)$ and $K_{**} = k(X_*, X_*)$ for the kernel evaluated between training/training, training/test and test/test points, the posterior predictive mean and covariance at test points X_* are

$$\bar{f}_* = K_*^T [K + \sigma_n^2 I]^{-1} y$$

$$\text{cov}(f_*) = K_{**} - K_*^T [K + \sigma_n^2 I]^{-1} K_*$$

where σ_n^2 accounts for observation noise. The predictive standard deviation at each test point is the square root of the corresponding diagonal entry of $\text{cov}(f_*)$: as illustrated in Figure 11, it shrinks near training points, where K_* correlates strongly with K , and grows far from them. Kernel hyperparameters (length-scale, noise level, smoothness ν for the Matérn kernel used here) are found by maximizing the log marginal likelihood of the training data, a non-convex optimization that in practice benefits from several random restarts.

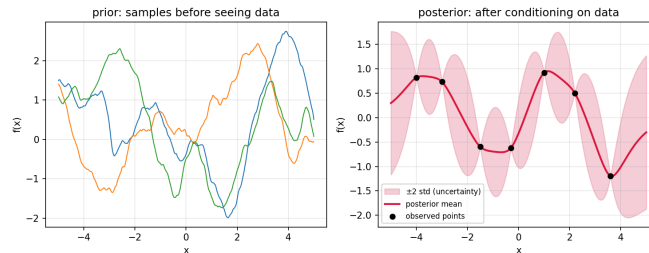


Figure 11: A Gaussian Process posterior mean (line) with its uncertainty band (variance): tight near observations, wide away from them.

Link to POD+GP: our own POD decoder, $\hat{\rho} = \bar{\rho} + \sum_m z_m \varphi_m$, follows the same idea: a fixed linear combination of basis fields whose coefficients are not fitted directly, but each predicted as the posterior mean $\hat{f}_*$ of their own GP over (M_∞, α, Π) , i.e. one GP per POD mode, each contributing its own posterior mean and variance.

A.4 Other Baselines and Hyperparameter Tuning

In addition to the two main baselines, the following models were also developed for comparison, and are accessible through the zip files on the root of the starting kit:

- **Global KNN regressor:** a $k = 5$ nearest-neighbor regressor fit on the scaled flow conditions (M_∞, α, Π) alone, one row per simulation, geometry is not used as an input at all. For a test condition, it predicts the **entire** wall ρ/ρ_∞ field as the average of its 5 nearest training simulations' fields. On Phase 1 it scores very well ($R^2 = 0.9824$, mean_KL= 1.314, Table 2), but this is largely interpolation in a favorable spot rather than genuine generalization: as Table 1 and Figure 14 show, the Phase 1 Mach values (0.84, 0.86) sit tightly bracketed by training values on both sides (0.82, 0.85, 0.88), all within the same dense cluster, and ρ/ρ_∞ varies smoothly with Mach there. The 5 nearest training simulations are therefore near-duplicates of the test condition, so this global KNN regressor is effectively **cheating** because it wins by proximity, not by having learned any structure. Phase 2 exposes exactly this: at $M_\infty \in \{0.30, 0.96\}$, far outside the training range, there is no nearby simulation to copy from, so mean_KL collapses to 7.410 and R^2 drops to 0.8480.
- **Mean predictor:** Naively predict the mean, just a starting point for the participants, to make sure that all the models are at least

better than the mean predictor.

- **IsoMap+RBF:** This model was selected from [2] as it scored the best results for predicting the C_p coefficient. Training fields are embedded into a 3-dimensional latent space via IsoMap (chosen to match the 3 physical parameters), and a Radial Basis Function (RBF) interpolator maps flow conditions to that latent space. Unlike POD's exact linear decoder above, IsoMap has no natural inverse, so reconstruction backmaps the predicted latent point via an inverse-distance-weighted average of the k nearest training fields in latent space. Because this backmapping is a convex combination of training fields, it cannot exceed the training range, a huge structural disadvantage for extrapolation. As a matter of fact, this model won the first phase of this challenge but was beaten by the POD+GP model in the second phase (see Section 4). A full Optuna search over all five of its hyperparameters was run (Section A.4.1) but did not clearly improve on the version from [2] (see Section A.5.2). The submitted-configuration numbers reported in Table 2 come from logs/438456.out.
- **Global MLP (literature architecture):** the architecture proposed in [2] (hidden layers of size 75, 120, 1226, 16490), trained with plain MSE rather than a KL-aware loss. This was the top-ranked model for predicting the C_p coefficient at the previous challenge; it was expected to work well for predicting the volumetric density. It scored very high in the first phase, but not as it should have in the second phase. This baseline was never submitted due to limitations on Codabench resources (logs/438440.out).

A.4.1 Tuning

POD+GP and IsoMap+RBF, being the best models that resolve the challenge, were tuned via Optuna's Bayesian (TPE) search, using leave-two-consecutive-Machs-out cross-validation restricted to the training set only.

POD+GP. The search covered the number of retained POD modes, the GP kernel smoothness $\nu \in \{0.5, 1.5, 2.5, \infty\}$, the regularization nugget α , and the kernel length-scale/noise-level bounds (see Section A.3), over 181 trials, pruned early on clearly bad configurations (Figure 12, left). Table 4 gives the best configuration found.

Hyperparameter	Value
Retained modes k	55
Kernel	Matérn($\nu = 1.5$) $\times$ const + white
Regularization α	7.49×10^{-5}
Length-scale bound	728.7
Noise-level bound	4.04
GP restarts	5

Table 4: POD+GP: best hyperparameters found by the 181-trial Optuna search (logs/438767.out).

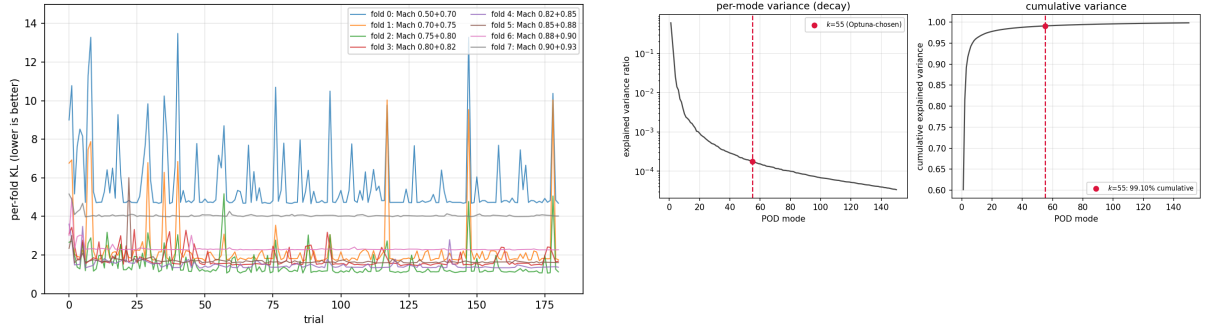


Figure 12: Left: POD+GP Optuna per-fold tuning curve (181 trials). Right: explained-variance profile of the retained POD modes, i.e. the cumulative sum of the training-field covariance’s eigenvalue spectrum, with the chosen $k = 55$ highlighted (logs/438767.out).

IsoMap+RBF. A separate 300-trial Optuna search (78 pruned, 222 completed) covered all five of the model’s hyperparameters: number of IsoMap components r , IsoMap neighborhood graph size, RBF kernel (linear/thin-plate/cubic/quintic), RBF smoothing, and backmapping k . The best trial found $r = 3$, graph size = 36, kernel = cubic, smoothing = 0.0119, backmapping $k = 4$, see Figure 13 for results.

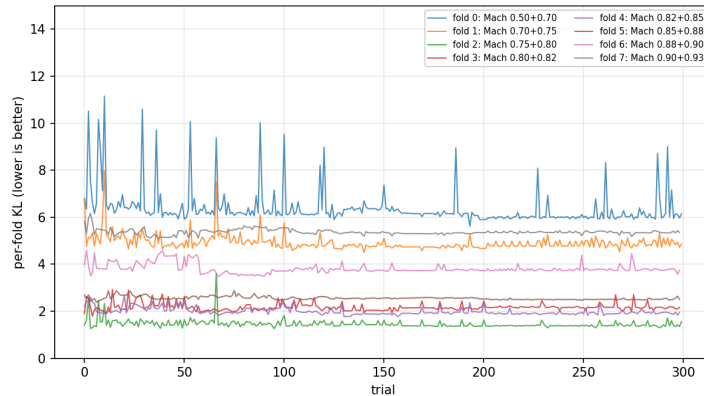


Figure 13: IsoMap+RBF: full 5-hyperparameter Optuna search, per-fold tuning curve (300 trials, 78 pruned) (logs/443009.out).

A.5 Results Diagnostics

A.5.1 Why did the initial global KNN regressor baseline fail?

The global KNN regressor baseline scores surprisingly well on Phase 1 ($R^2 = 0.9824$, $\text{mean_KL} = 1.314$, Table 2), despite never seeing the aircraft’s geometry. Phase 1’s test Mach values (0.84, 0.86) sit tightly bracketed by dense training values (0.82, 0.85, 0.88) (see Figure 14), so the global KNN regressor simply retrieves near-exact matches.

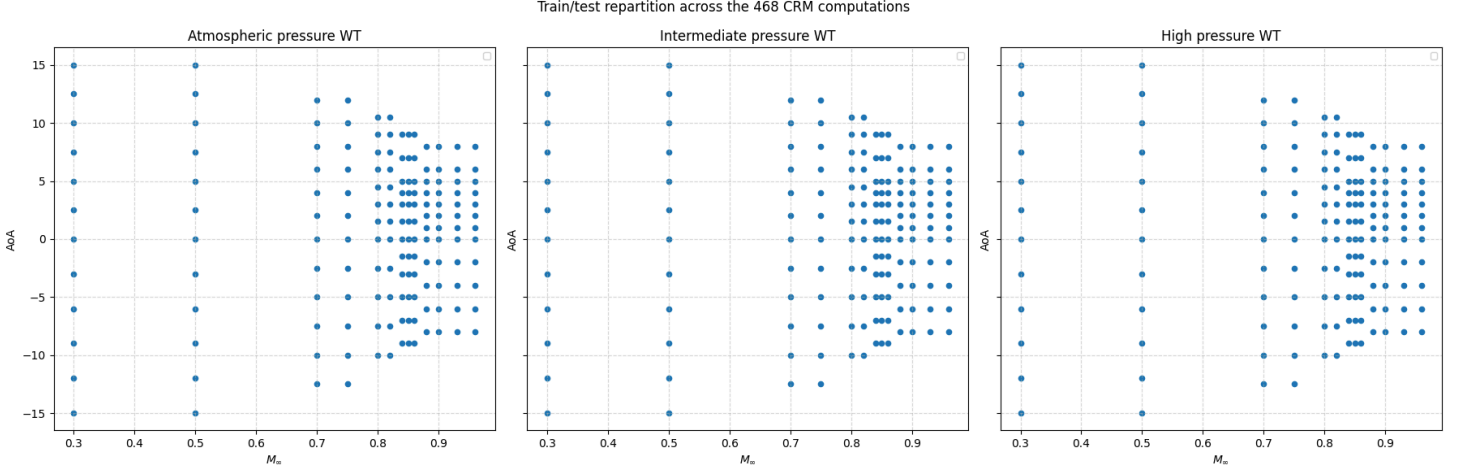


Figure 14: Illustration of the distribution of the flow conditions.

Phase 2 exposes the shortcut, at $M_\infty \in \{0.30, 0.96\}$, far outside the training range $[0.50, 0.93]$, mean_KL collapses to 7.410 and $R^2 = 0.8480$ (see Table 2). With no nearby training points, the global KNN regressor averages dissimilar fields instead of interpolating near-identical ones. It never learned the true dependence of ρ/ρ_∞ on M_∞ , α , and Π , only how to look up the closest match. This is the general failure mode of baselines that exploit local smoothness rather than genuine structure, and it’s one of the points that Phase 2 is designed to expose.

A.5.2 Why POD+GP outperforms IsoMap+RBF in extrapolation

Two things stand out in Table 2. First, Global MLP and IsoMap+RBF both edge out POD+GP on Phase 1 (interpolation), but POD+GP has the best Phase 2 (extrapolation) mean_KL of every baseline, submitted or not. This is consistent with the mechanism described in Section 3: POD+GP’s decoder is exact and linear, whereas IsoMap+RBF’s is a convex combination of training fields that cannot, by construction, extrapolate past the training range. Since Phase 2 is what the final leaderboard rank is decided on, this makes POD+GP the strongest baseline overall.

In addition to the decoder difference, the Gaussian Process itself contributes to this gap: swapping it for a plain RBF interpolator while keeping the same POD basis (POD+RBF, the configuration used in [2]’s own baseline, Section 5.3) consistently underperforms POD+GP. The POD+GP model was actually inspired by the top-ranked model of the NeurIPS 2024 ML4CFD competition, which treats a slightly different problem; this model is MMGP (Mesh-Morphing Gaussian Process) [6], which similarly uses a GP. Apparently GP models are a good choice to tackle this kind of challenge.

A.5.3 Principle of the error-weighted PCA slices

The u, v slice curves used in the diagnostics below (computed only on the wings part) were designed for the participants to use and evaluate their models more easily; they might be very helpful in spotting the regions where the model fails, and they answer one question: **Which is the direction on the aircraft’s skin where the error is mostly scattered, and how is the error evolving**

across this direction? This direction is computed by using a PCA, with error weighting. Ordinary PCA on the surface geometry would just recover the aircraft's own shape (wingspan, chord). Instead, each point i is weighted by its own error, $w_i = |\hat{\rho}_i - \rho_i|^2$, before computing the principal directions:

$$\mu = \frac{\sum_i w_i X_i}{\sum_i w_i}, \quad C = \frac{\sum_i w_i (X_i - \mu)(X_i - \mu)^T}{\sum_i w_i}$$

where $X_i \in \mathbb{R}^3$ is point i 's surface coordinate. Because large-error points now dominate this weighted covariance C , its leading eigenvector u (largest eigenvalue) points along whatever direction the **error** is most spread out, typically a shock line or separation front, rather than along the wing itself. The next eigenvector v is orthogonal to it, cutting across that structure. Projecting each point onto (u, v) , we take thin slices at fixed u values where the error across v is maximal, then let us plot ρ (true) against ρ (predicted) along v both on the intrados and the extrados which gives exactly the cross-section through the hardest, most error-concentrated region of the surface.

If we instead plotted directly across u , the result would be unclear: many points can project onto the same u value, so a curve $\rho(u)$ would not be well defined. By fixing u at the positions where the error is most concentrated and plotting across v instead, at most two points project onto each v value, one on the intrados and one on the extrados, since a thin slice at fixed u crosses the surface at exactly two points along its thickness. Plotting these two as separate curves therefore gives a clean, well-defined error analysis.

Generally, the error is scattered across the wing, so for example, the plot with the lowest u value often refers to the variation of ρ along the small part representing the wing's extremity, where very particular aerodynamic phenomena occur, making it hard to predict and associated with the curve at the lowest u value.

A.5.4 Per-model diagnostics: POD+GP

Figure 15 shows, for the worst-KL condition on each phase, the ground truth field, the POD+GP prediction, and the absolute error painted on the aircraft surface. Figure 16 and Figure 17 show, for the same two conditions, the error-weighted PCA slices.

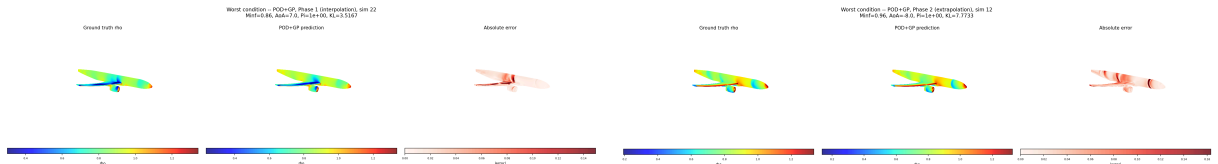


Figure 15: POD+GP: worst-KL condition, ground truth / prediction / absolute error. Left: Phase 1. Right: Phase 2. Flow conditions are given in each panel's own title.

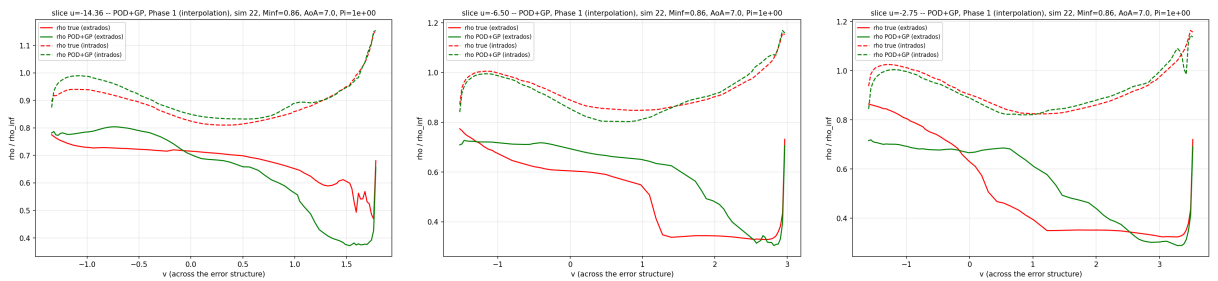


Figure 16: POD+GP, Phase 1: error structure, slices along v .

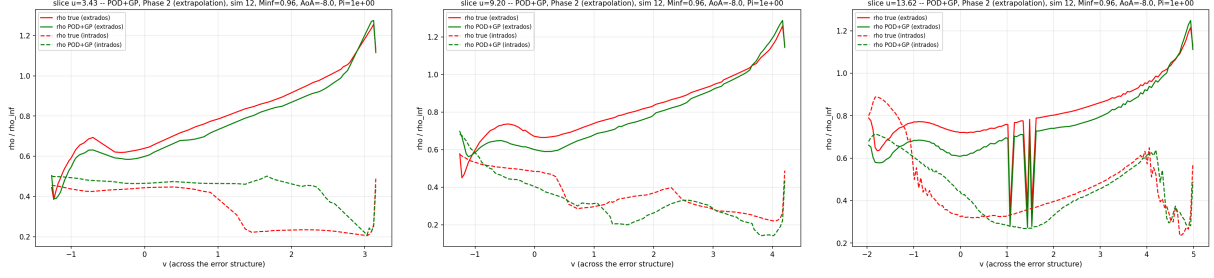


Figure 17: POD+GP, Phase 2: error structure, slices along v .

According to the error-weighted PCA plots, we can clearly observe that the POD+GP model works very well on the intrados, better than on the extrados, especially in Phase 1. We can also observe that it is able to capture the discontinuities present in Figure 17, on the right, for $v \in [0.8, 2.0]$. This is a good sign of extrapolation and a good indicator that our model has a respectable understanding of the underlying physics that govern this part of the extrados, which corresponds to $u = 13.62$. After plotting the direction of the arrows, one can notice that it corresponds to the edge between the wing and the fuselage, which is not an easy part to predict.

A.5.5 Per-model diagnostics: MLP_{KLW}

The same diagnostics for MLP_{KLW} are given in Figure 18, Figure 19, and Figure 20, using the same worst-KL-condition selection and slice-placement procedure as Section A.5.4.

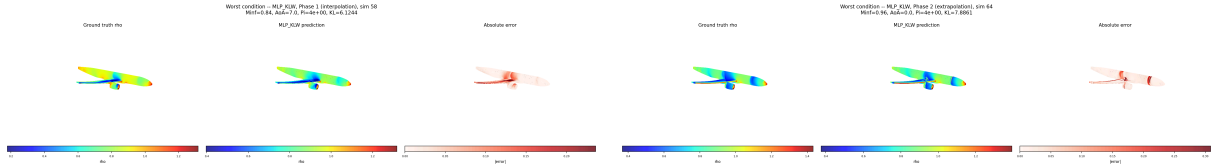


Figure 18: MLP_{KLW} : worst-KL condition, ground truth / prediction / absolute error. Left: Phase 1. Right: Phase 2.

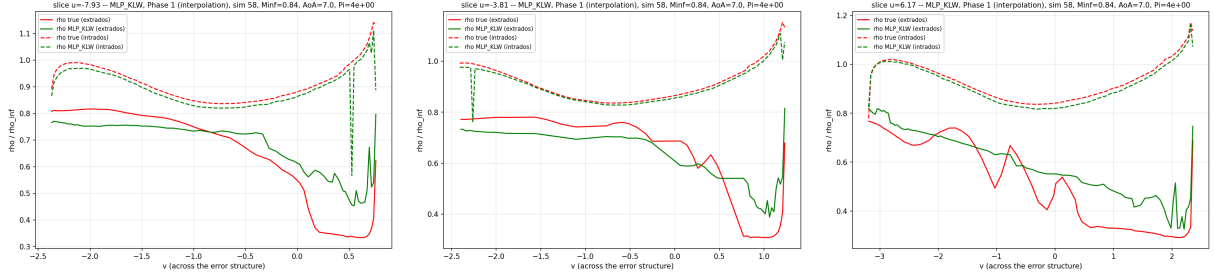


Figure 19: MLP_{KLW} , Phase 1: error structure, slices along v .

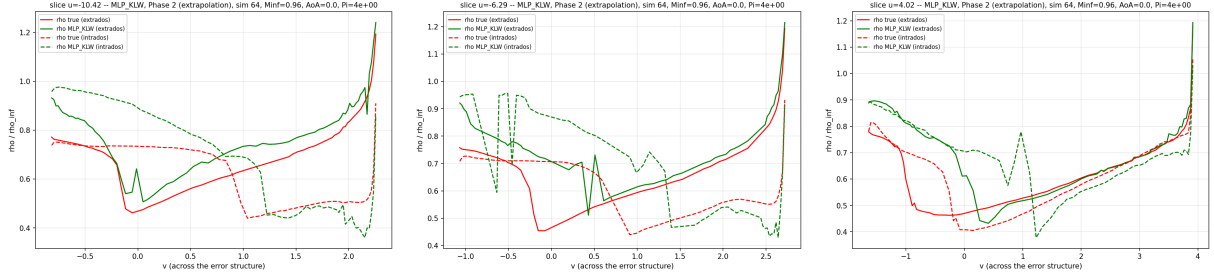


Figure 20: MLP_{KLW} , Phase 2: error structure, slices along v .

Globally, we can observe that the evolution of the green lines is zigzagging and has more variance with respect to the ground truth, both for the intrados and for the extrados. This is a sign of overfitting, which means our model, even with the different techniques used to prevent it, still falls

into that trap. Participants may further improve the model on this front by employing fine-tuning techniques. However, overall, we can notice a respectable amount of similarity between the green and red curves, which is a good indicator that our model is starting to have a respectable understanding of the underlying physics, and could be improved significantly further on this front.

A.6 Competition Platform

The challenge is hosted on the Codabench platform [3] and follows a two-phase structure: a **Development phase** (Phase 1, interpolation), which is feedback-only, allows up to 100 submissions, and never counts toward the final ranking, and a **Final phase** (Phase 2, extrapolation), to which participants must actively resubmit their model, nothing carries over automatically from Phase 1, and which is what actually determines the final rank.

The competition page itself is organized into four tabs: **Overview** (physics and problem statement), **Data** (file format, confidence weights, split description), **Evaluation** (metric definitions and leaderboard), and **Extra packages** (instructions for bundling dependencies beyond the base Docker image). A submission is a zip archive containing a single `model.py` at its root, exposing a `Model` class with `fit(X, y)` and `predict(X)` methods; `fit()` is re-run from scratch on every submission and must complete inside the platform’s time limit. Five ready-made alternative submissions, covering the baselines of Section 3, ship in the starting kit as a reference, one submission showing an example of using packages that are not present on the docker image, and another submission showing how to upload an already-trained model with pytorch saved as a `.pt` file to use directly without wasting time or resources for training on codabench. The leaderboard displays **mean_KL** as the primary, rank-determining column (ascending order, lower is better), with Score, R^2 , wrMAE, and Duration shown alongside for context only.

The challenge is live on Codabench at codabench.org/competitions/17632. A Discord server, discord.gg/kdP863nY7, is also available and can be used in place of the Codabench forum to form teams and discuss any issues that come up; it includes a roles section letting participants tag themselves by profile (researcher, PhD student, Master’s student, etc.) to more easily find and team up with others like them. The code behind the challenge (baselines, tuning scripts, split generation, and this report) is available at github.com/Akawalid/onera_468_crm_wall_distribution_regression_challenge_v2.

Bibliography

- [1] NASA, “NASA/Boeing Common Research Model (CRM) Aerodynamic Database.”
- [2] J. Peter, Q. Bennehard, S. Heib, J.-L. Hantrais-Gervois, and F. Moëns, “ONERA’s CRM WBP database for machine learning activities, related regression challenge and first results,” *Computers & Fluids*, vol. 302, p. 106838, 2025, doi: 10.1016/j.compfluid.2025.106838.
- [3] “Codabench: Flexible, Easy-to-Use, and Reproducible Benchmarking Platform,” *Patterns*.
- [4] “AirfRANS: High Fidelity Computational Fluid Dynamics Dataset for Approximating Reynolds-Averaged Navier-Stokes Solutions,” in *NeurIPS 2022 Datasets and Benchmarks Track*, 2022.
- [5] A. Pavão, I. Guyon, E. Viegas, and others, *AI Competitions and Benchmarks – The Science Behind the Contests*. Submitted at DMLR, 2025. [Online]. Available: <https://ai-competitions-book.github.io/>
- [6] M. Yagoubi *et al.*, “NeurIPS 2024 ML4CFD Competition: Results and Retrospective Analysis,” *arXiv preprint arXiv:2506.08516*, 2025, [Online]. Available: <https://arxiv.org/abs/2506.08516>

- [7] J. Wang, “An Intuitive Tutorial to Gaussian Process Regression,” *arXiv preprint arXiv:2009.10862*, 2023, [Online]. Available: <https://arxiv.org/abs/2009.10862>